C# allows you to create overloads of methods by varying the number and type of parameters they accept. While it is possible to create multiple versions of the `OnGet` or `OnPost` methods that vary by parameters in a single `PageModel` class and have them compile successfully, the Razor Pages framework does not allow it. You can only have one `OnGet` or one `OnPost` method in a `PageModel`. In fact, you can't even have an `OnGet` and an `OnGetAsync` method in the same `PageModel` class. Razor Pages ignores the `Async` suffix when it matches a handler to an HTTP method name. If more than one handler matches a given request, you will get an `AmbiguousActionException` at run time.

There may be times you want different code to execute for the same HTTP method. For example, you might have a page with multiple forms in it. The Manning home page features both a search form and a registration form. One takes a search term, and the other one takes an email address. Assuming they both post back to the same page, how will you know which one the user submitted? You could add some logic in the `OnPost` handler to try to identify whether the user was attempting to register for a newsletter or searching for an author by their email address—or you can use *named handler* methods.

Named handler methods start with the same convention as standard handler methods: `On` followed by the HTTP method name. This is followed by the *name* of the handler method and is used to disambiguate it from standard handler methods and other named handler methods. For example, you might want to create a method named `OnPostSearch` to process a search form submission and another named `OnPost- Register` to process the registration form submission. The following listing shows how you would implement these in the `WelcomeModel` of your sample application.

Listing 3.31 Showing two named handler methods

```
public class WelcomeModel : PageModel
{
    public string Message { get; set; }                    ❶
    public void OnPostSearch(string searchTerm)            ❷
```